

Algo-Scope 기획안 (설계 문서) v3.1 — 최종

- 작성일: 2026-08-11 (v3 기획 동결 + v3.1 지원 범위 매트릭스 명문화)
- 팀: 컴퓨터공학 전공 3인
- 상태: **기획 동결**. 이후 변경은 스파이크 실측 결과에 의한 계약 수정만 허용

0. 한 문장

The trace decides what happened; the Director decides what is worth showing. 무슨 일이 일어났는가는 실행이 결정하고, 그중 무엇을 어떻게 보여줄지는 AI가 결정한다.

1. 제품 정의

기술적 정의: Python 코드의 실제 실행 상태를 추적하고, 이를 자동 설명 애니메이션(무비)으로 변환하는 코드 실행 시각화 도구.

제품 포지셔닝: 특히 AI(ChatGPT·Claude 등)가 생성한 코드를 자기 프로젝트에 적용하기 전에 실행 흐름을 이해하는 용도. "AI 생성 코드"는 기술적 제약이 아니라 대표 사용 시나리오다 — 사람이 쓴 코드도 동일하게 처리한다.

해결하는 문제: AI 코드를 이해 없이 복붙하는 습관. AI의 텍스트 설명만으로는 실행 흐름이 머리에 그려지지 않는 학습자가 많다.

핵심 제품 가설 (실험으로 검증할 대상이며, 전제가 아니다):

실제 실행 상태를 시간적으로 시각화하면, 텍스트 설명만 제공하는 것보다 프로그램 상태 변화와 실행 흐름을 이해하는 데 도움이 된다.

차별화 3축:

1. **값의 신뢰성** — 화면에 표시되는 모든 값·순서는 실제 실행에서만 나온다. 단, 이것은 "해당 실행에서 실제로 발생한 상태를 보여준다"는 뜻이지 코드가 논리적으로 올바르다는 보증이 아니다.
2. **재생 모델** — Python Tutor가 사용자가 직접 스텝을 탐색하는 debugger-first 경험이라면, 본 도구는 중요한 장면·속도·표현을 골라 자동 재생하는 narrative-first 경험이다. 핵심 축: Debugger → Movie.
3. **영상 그 이상** — 실시간 렌더이므로 언제든지 일시정지하고 그 순간의 변수·객체 상태를 들여다볼 수 있다.

Core와 Quality의 분리 (심사 방어용 논리 구조):

- **Core (제품 완주 조건):** 실제 trace + trace-grounded rendering + 자동 재생 + 일시정지 인스펙터. LLM 없이 규칙 기반 대본만으로 성립한다.
- **Quality layer (품질 차별 조건):** LLM Director가 중요한 장면과 표현을 선택해 규칙 기반 무비보다 설명 품질을 높인다. "왜 AI가 필요한가"는 ablation 평가(11절)로 정량 증명한다.

2. 범위 — 증명 범위(MVP)와 확장 범위의 분리

완성형 비전과 MVP를 혼재시키지 않는다. 아래 분리는 일정이 아니라 우선순위 정의다.

증명 범위 (MVP)

- **지원 코드:** self-contained · single-file · synchronous Python (표준 라이브러리 중심)
- **명시적 범위 밖:** 외부 모듈·DB·네트워크·프레임워크 의존 코드, `input()` 등 interactive code, generator/coroutine/async/thread. 실행 전 preflight에서 감지 시 안내하며, **벤치마크 샘플 셋에서도 제외**한다 (지원하는 척하지 않는다)
- **시각화:** 프리미티브 5종 + Generic State View
- **렌더러:** SVG + GSAP 단일

확장 범위 (MVP 안정화 이후)

- generator/coroutine (TraceEvent에 yield/resume semantics 추가 필요), `input()` (입력 수집 UI + stdin 이벤트 필요), 프리미티브 추가(호출 트리, 표, 대량 배열), PixiJS 대량 연출, 서버 실행기(하이브리드), 언어 확장

지원 범위 매트릭스

"어떤 코드가 들어오면 어떻게 되는가"를 판정별로 명문화한다. 제품 문서·심사 답변·preflight 구현의 공통 기준이다.

A. 완전 지원 — 특화 시각화로 재생

코드 유형	예시	화면
변수 할당·연산	<code>x = a + b</code>	변수 패널, 값 변화 하이라이트
조건문·반복문	<code>if / for / while</code>	코드 하이라이트, 루프 접기·빨리감기
함수 호출·재귀	<code>f(x)</code> , 재귀 호출	호출 스택 카드
리스트·dict·set 조작	<code>arr.append(3)</code> , <code>d["k"] = v</code>	시퀀스 상자, mutation 애니메이션
aliasing·얕은 복사	<code>b = a</code> , <code>b = a[:]</code>	객체 참조 그래프 (같은 객체 화살표)
예외 발생	<code>arr[5] → IndexError</code>	터지는 지점까지 재생 + 하이라이트 (콘텐츠로 취급)
클래스 인스턴스 기본	<code>p = Point(1, 2)</code>	객체 상자 (필드 표시)
컴프리헨션·문자열 처리·print	<code>[x*2 for x in arr]</code>	시퀀스·콘솔 출력

B. 지원 — Generic State View로 표시 (전용 그림 없음)

코드 유형	예시	화면
트리·그래프 등 복잡한 사용자 정의 구조	<code>Node(1).children = [...]</code>	변수/타입/값/변경 여부 표 (확장 범위에서 전용 프리미티브 추가 예정)
깊은 중첩 데이터	<code>{"a": {"b": [{...}]}}</code>	제한 깊이까지 표 형태

C. 실행되지만 제한 — 상한까지만 재생

코드 유형	예시	동작
매우 긴 실행	10만 회 루프	3중 상한(이벤트 수/직렬화 크기/시간)까지 수집 후 "여기까지 시각화"
무한 루프	<code>while True:</code>	타임아웃 강제 종료, 수집분까지 재생 (chunk flush로 보존)
대량 데이터	원소 10만 개 리스트	bounded serializer가 앞부분만 표시

D. 범위 밖 — 실행 전 preflight에서 감지·안내 (실행하지 않음)

코드 유형	예시	안내 문구 방향
사용자 입력	<code>input()</code>	"입력 대기 코드는 아직 지원하지 않아요"
generator/async/thread	<code>yield, async def, threading</code>	"이 실행 모델은 아직 지원하지 않아요"
네트워크	<code>requests.get(...)</code>	"브라우저 실행 환경에서는 네트워크에 접근할 수 없어요"
파일·DB	<code>open("f.txt")</code> , DB 드라이버	동일 계열 안내
미지원 외부 라이브러리	<code>import django</code>	"표준 라이브러리 중심 코드를 지원해요"
조각 코드 (미정의 이름)	<code>self.repository.find(...)</code>	"독립 실행 가능한 형태로 잘라서 넣어주세요"
정의만 있고 호출 없음	함수 정의만	"실행이 없어요 — 호출 예시를 한 줄 추가해 주세요 (예: <code>print(f([3,1,2]))</code>)"

- preflight는 best effort다(7절): D를 놓치고 실행에 들어가도 런타임 에러 처리로 안전하게 내려온다.
- 이 매트릭스는 벤치마크 샘플 셋의 총화 기준과 일치시킨다 — A·B·C는 측정 대상, D는 안내 문구의 정확성만 측정한다.

3. 시스템 아키텍처 — 4단계 파이프라인

코드 붙여넣기

- ① Tracer : Pyodide(Web Worker) 실행 + 상태 스냅샷·비교 → TraceEvent 스트림
- ② Digest : 프레임 필터링, 루프 접기, 압축 → Digest + DigestSpan
- ③ Director : LLM이 코드+Digest를 읽고 연출 대본(Screenplay JSON) 생성 [검증 + 결정적 리졸버]
- ④ Player : 대본+트레이스 → GSAP 마스터 타임라인 → SVG 렌더

성격 규정: Execution-grounded / Non-generative. 사용자 코드 자체는 `random · time` 등으로 비결정적일 수 있으므로 "같은 트레이스가 주어지면 Digest·Player의 출력이 결정적"이라고 기술한다.

실행 위치와 데이터 흐름: 코드 실행과 시각화(①②④)는 브라우저에서 처리하고, 연출 생성(③)에서만 서버리스 프록시를 통해 외부 LLM API를 호출한다. 이때 **코드와 Digest가 외부로 전송된다** — 사용자 고지, opt-out(규칙 기반 폴백만 사용), local-only 모드를 제품에 반영한다.

LLM의 지위: core execution engine이 아니라 quality enhancement layer. 규칙 기반 대본 생성기만으로도 제품은 완주한다 (Slice 1이 이를 증명).

보안 — untrusted code 위험 모델

이 제품의 사용자는 **이해하지 못한 코드를 실행하러 온다**. 즉 의도적으로 신뢰할 수 없는 코드를 실행하는 서비스이며, Web Worker는 UI 스레드 격리일 뿐 네트워크 접근·JS bridge 문제를 자동으로 막지 않는다. 설계 원칙:

- 실행 Worker에 애플리케이션 secret을 절대 두지 않는다 (LLM API 키는 서버 측에만 보관)
- CSP/connect-src로 실행 컨텍스트의 네트워크 접근을 제한한다
- 실행 컨텍스트와 인증 컨텍스트를 격리한다
- Pyodide의 JS bridge 접근 제한 가능성은 스파이크로 검증한다
- 표현: "사용자 코드를 서버에서 직접 실행하지 않으므로 서버 측 원격 코드 실행 위험과 인프라 비용을 크게 줄인다" ("보안 문제 제거"라고 주장하지 않는다)

모듈 경계

모듈	책임	입력 → 출력
Tracer	코드 실행, 상태 스냅샷·비교, 이벤트 합성	코드 문자열 → TraceEvent[] (chunk 단위 flush)
Digest	트레이스 압축·요약, 스패ن 매핑	TraceEvent[] → Digest + DigestSpan[]
Director	연출 대본 생성	코드 + Digest → Screenplay JSON (spanRef 기반)
Resolver	대본의 참조를 트레이스 구간으로 결정적 치환	Screenplay + DigestSpan[] → 실행 가능한 대본
폴백 생성기	LLM 없이 규칙만으로 대본 생성	TraceEvent[] → Screenplay JSON
Player	타임라인 조립·렌더·인스펙터	치환된 대본 + TraceEvent[] → 재생 가능한 무비

각 모듈은 위 입출력 계약으로만 통신한다. 내부 구현을 바꿔도 옆 모듈이 깨지지 않아야 한다.

4. 데이터 계약 — 스키마 3개

TraceEvent

```
{ seq,
  kind: line | call | return | mutation | exception,
  frameId, parentId, function,           // 재귀·동일 함수 반복 호출 구분
  causedByLine, observedAtLine,         // 상태 변화 귀속 규칙 (아래)
  localsDelta: [ { name, op: set | delete, value } ],
  reachableObjectsDelta: [ { objectId, type, op: set | delete, fields } ],
  stdout }
```

- **스텝과 귀속 규칙**: line 이벤트는 해당 줄을 실행하기 직전에 발생한다. 따라서 source-line boundary 기준으로 pre-state를 수집하고, 다음 boundary에서 얻은 post-state와의 diff를 **직전에 실행된 줄 (causedByLine)의 효과**로 귀속한다. 제품이 보여주는 것은 "모든 mutation"이 아니라 "**source-line boundary에서 관측되는 상태 변화**"다 — 한 줄 안에서 변했다가 원상복귀된 중간 상태는 관측 대상이 아니며, 이를 문서·제품 문구에서 정직하게 표현한다.
- **값 표현**: primitive 값과 objectRef(objectId 참조)를 구분한다. aliasing은 objectId 공유로 표현된다. reachableObjectsDelta는 Python 힙 전체가 아니라 사용자 프레임 변수에서 도달 가능한 객체 중 시각화에 필요한 부분만을 뜻한다. del·필드 삭제는 op: delete로 표현한다.
- **bounded serializer — 부작용 없음 원칙**: depth·길이 제한, cycle detection에 더해 **observationally passive**해야 한다. repr()·getattr()·property 접근은 사용자 코드를 실행할 수 있으므로, 안전한 built-in 타입(primitive, list/tuple, dict, set)을 우선 지원하고 일반 클래스는 __dict__ 등 부작용 적은 경로로 제한 관찰하며, 불확실한 객체는 UnsupportedObject로 축약한다. 관찰이 프로그램 동작을 바꾸면 안 된다.

- **mutation은 합성 이벤트**: 실행 혹(line/call/return/exception)이 주는 것이 아니라 Tracer가 상태 비교로 직접 합성한다. 이 상태 직렬화 계층이 본 프로젝트 최대의 기술 난제다 (리스크 1순위).

DigestSpan (Digest ↔ 원본 Trace의 안정 참조)

```
{ spanId, sourceSeqRange: [시작, 끝], line, iterations,
  changedVariableRefs, objectRefs, eventKinds, exception?, stdoutDelta }
```

- Digest의 각 요약 단위가 원본 트레이스의 어느 구간인지 고정하는 계약. Director가 연출 판단에 필요한 요약 정보(변한 변수, 관련 객체, 이벤트 종류, 예외, 출력)를 충분히 포함한다.
- 루프 접기는 line 패턴 반복만이 아니라 **state 변화 패턴**을 함께 비교해 boring iteration과 interesting iteration을 구분한다 — Digest의 책임. interesting iteration은 별도 스패스로 분리해 Director가 세밀하게 참조할 수 있게 한다.

Screenplay (연출 대본)

```
{ chapters: [
  { title, // 해석 성격 (아래 신뢰 구분 참조)
    scenes: [
      { spanRef, // LLM은 seq 숫자를 직접 쓰지 않는다
        primitive,
        focus: [ variableRef { frameId, name } | objectRef { objectId } ],
        pacing: normal | slow | fastForward(배속),
        narration: { template, bindings }, // 값 문자열 직접 작성 금지
        camera } ] } ] }
```

- **spanRef 원칙**: Director는 seq 숫자를 생성하지 않고 DigestSpan의 안정 ID를 선택만 한다. **결정적 Resolver**가 spanRef를 sourceSeqRange로 치환하고 실행 순서를 보장한다. LLM이 실행 사실(순서 포함)을 만들 경로가 구조적으로 사라지고 검증도 단순해진다.
- **narration의 신뢰 구분**: 관찰형 서술(값·상태·호출·예외 — 템플릿+바인딩으로 trace가 강하게 보장)과 해석형 서술(변수의 역할, 왜 중요한지 — LLM 판단, 오류 가능)을 계약에서 구분한다. **장면 narration은 관찰형 중심**으로 하고, 해석형은 챕터 제목·장면 요약 수준으로 한정한다. 해석의 품질은 ablation(11절)의 측정 대상이다.
- **검증**: (1) JSON Schema 구조 검증 → (2) 참조 실존 검증(spanRef-variableRef-objectId가 실제 존재) → (3) Resolver 치환. 불통과 시 1회 재시도, 재실패 시 규칙 기반 폴백.

5. 시각 언어

프리미티브 — MVP 5종

1. 코드 하이라이트 + 내레이션 자막 — 상시 표시, 현재 실행 줄(causedByLine)과 자막 동기화
2. 변수·함 패널 — 프레임별 변수와 값
3. 호출 스택 — 쌓이는 카드, 카드 안에 지역변수
4. 시퀀스(리스트) — 칸이 나뉜 상자, 인덱스·커서 표시
5. 객체 참조 그래프 — 이름표(변수) → 참조 화살표 → 객체 상자. 별칭·얕은 복사 표현의 핵심

Generic State View

특화 프리미티브가 없는 값도 최소한 Variable / Type / Value / Changed 구조로 항상 표시한다. **지원 범위 내에서** 특화 시각화가 없는 상태를 표현하기 위한 최종 안전망이다 (async.thread.native module 등 범위 밖 실행 의미까지 지원한다는 주장이 아니다).

연출 규칙

- 동시에 한 동작만. 나머지 요소는 흐리게(dim) 처리
- 상태 색 체계: 기존 프로토타입의 상태색 계승
- easing 표준: 등장 back.out, 이동 power3.out, 스왑 elastic.out
- 중요한 순간은 슬로우, 반복 루프는 "xN회 반복" 표시와 함께 빨리감기
- 내레이션은 바인딩된 실제 값으로 "지금 화면에서 일어난 일"을 서술

렌더러와 semantic timeline

- MVP는 **SVG + GSAP 단일 렌더러**. 기존 PixiJS 스테이지는 보류 자산으로 두고, 벤치마크에서 대량 배열 성능 병목이 실측된 경우에만 확장 범위에서 투입한다.
- **semantic timeline**: GSAP의 물리적 애니메이션 시간과 "마지막으로 commit된 Trace seq"를 분리해 매핑한다. 애니메이션 중간에 일시정지해도 인스펙터는 항상 특정 seq의 일관된 상태를 보여준다.

6. 재생 경험

1. 코드 붙여넣기 → Run
2. **로딩**: 파이프라인 실제 진행 단계를 표시 — "코드 실행 중 → 실행 기록 분석 중 → 반복 구간 감지 → 설명 준비 중" (챕터 목차는 Director 완료 후에만 표시 가능)
3. 자동 재생 — 챕터 구분 진행바(유튜브 챕터식), 내레이션 자막 동기화
4. 인터랙션: 일시정지 / 스크립 / 배속 / 챕터 점프. 일시정지 시 변수 인스펙터로 해당 seq 상태 조회
5. 레이아웃: 좌측 코드 에디터(Monaco) / 우측 스테이지 — 현 프로토타입 계승

지연 예산: 콜드/웜 스타트를 구분하고 Tracer·Digest·Director·Player 준비·E2E 각각 P50/P95를 벤치마크로 측정한다. 콜드 스타트는 페이지 진입 시 백그라운드 선로딩으로 흡수한다. **합격 기준 수치는 지금 임의로**

정하지 않고, Tracer 스파이크에서 baseline을 실측한 뒤 MVP acceptance criteria로 고정한다 (지표 항목은 11절에 확정).

7. 에러 처리와 실행 제한

상황	처리
사용자 코드가 예외로 종료	실패가 아니라 콘텐츠. 예외 발생 지점까지 재생하고 터지는 순간을 하이라이트
LLM 실패·검증 불통과(재시도 후)	규칙 기반 대본 생성기로 폴백 — Core 기능은 LLM 없이도 유지
특화 프리미티브 매핑 실패	Generic State View로 값 단위 폴백
실행 제한 초과	maxEvents + maxSerializedBytes + maxRuntime 3중 상한. 초과 시 수집분까지 "여기까지 시각화"
무한 루프·중단	이벤트를 chunk 단위로 main thread에 flush — Worker terminate에도 수집분 보존. 정상 interrupt(SharedArrayBuffer + COOP/COEP 헤더 — 배포 환경 포함 스파이크 검증)와 terminate 경로를 구분 설계
실행 불가 코드 감지	정적 preflight는 best effort (동적 import-getattr 때문에 불완전) + 런타임 에러 처리의 2단계. 범위 밖 기능(input, generator 등)은 preflight에서 안내

8. 해야 할 일

구현 과제

과제	내용
스키마 확정	TraceEvent·DigestSpan·Screenplay JSON 스키마 정의 및 버전 관리
Tracer	Pyodide + Web Worker 실행, 스텝 귀속 규칙 구현, observationally passive serializer, 이벤트 합성, chunk flush, 3중 상한
Digest	프레임 필터링, state 기반 루프 접기(interesting iteration 분리), DigestSpan 생성
폴백 생성기	LLM 없이 규칙만으로 대본 생성 — Director보다 먼저 구현 (Slice 1의 주역)
Director + Resolver	프롬프트 설계, spanRef 기반 대본 생성, 검증·재시도, 결정적 치환, 캐싱 (키: code + digestHash + schemaVersion + promptVersion + modelVersion)
Player	GSAP 타임라인 조립, semantic timeline, 재생·스크립·배속·챕터 점프, 일시정지 인스펙터
프리미티브 5종 + Generic State View	SVG 컴포넌트로 구현
UI	에디터·스테이지·진행바·인스펙터, 파이프라인 진행 로딩, 외부 전송 고지·opt-out
보안 구성	Worker 무비밀 원칙, CSP/connect-src, LLM 프록시(키 서버 측 보관)
fixture 체계	계약 테스트의 중심. 최소 포함: 단순 할당, for/while, 함수 호출, 재귀, list/dict mutation, aliasing·얕은 복사, 예외, 중첩 객체, 클래스 인스턴스

검증 과제

과제	확인하려는 것
Tracer 스파이크 [최우선]	settrace vs monitoring을 상태 수집 중심 기준 으로 Go/No-Go: 현재 frame 특정 가능성 / f_locals 정확성 / 재귀·반복 호출에서 frame identity 유지 / line 단위 스냅샷 오버헤드 / 1만·10만 이벤트에서 메모리·직렬화 비용 증가. 판단 원칙: 성능이 다소 느려도 상태 수집이 단순·안정적이면 settrace 채택 가능
보안 스파이크	Pyodide JS bridge 접근 제한 가능성, interrupt(SharedArrayBuffer/COOP-COEP)의 배포 환경 성립 여부
대본 품질 스파이크	LLM이 실제 Digest로 검증을 통과하는 spanRef 대본을 안정적으로 내는가
기술 벤치마크	유형별 증화 샘플 셋(제어 흐름/함수/재귀/mutation/aliasing/중첩 데이터/예외/클래스/컴프리헨션 — generator-input 제외) → Tracer 정확성, Digest 압축률, 대본 검증 통과율, 폴백률, 렌더 완주율, E2E 지연을 유형별 측정. 스�파이크 baseline 측정 후 각 지표의 합격 기준을 수치로 고정
Ablation: 규칙 무비 vs LLM 무비	LLM이 장식이 아니라 실제 가치를 더하는가 — 중요 장면 선택 정확성, 불필요 스텝 감소율, 설명 길이·복잡도, 사용자 이해도, 시청 완료 시간 비교. "왜 AI가 필요한가"에 대한 정량 답변
예비 이해도 실험	효과 검증이 아니라 절차 검증 — 문항 난이도, 인터페이스 이해도, 소요 시간, 로그 수집 방식 조정 (N=6~10)
이해도 비교 실험	확보 가능한 표본 수를 먼저 계산 한 뒤 설계 선택: 3조건(LLM 텍스트/Python Tutor/본 도구) between-subject, 2조건, 또는 crossover/within-subject. 문항은 값 기억이 아니라 transfer question(입력이 바뀌면 결과·참조 관계 예측) 중심. 표본이 작으면 exploratory study로 표현

착수 순서 (일정 아님 — 의존 순서)

1. **즉시 병렬 시작**: Slice 0 (수작업 fixture → Player, 렌더 계약·semantic timeline 검증) || Tracer 스파이크
2. **Slice 1**: 실제 Tracer → 규칙 기반 대본 → Player. LLM 없이 E2E 완주 (Core 증명)
3. **Slice 2**: Tracer → Digest → LLM Director → Player. 품질 계층 추가, ablation 측정 시작
4. 이후: 프리미티브·커버리지 확장, 벤치마크 목표치 달성, 이해도 실험

9. 역할 분담 (3인)

역할	담당 모듈	주요 기술
실행·트레이스 담당	Tracer + Digest + 보안 스파이크	Pyodide, Web Worker, settrace/monitoring, 상태 직렬화
LLM·대본 담당	Director + Resolver + 폴백 생성기 + 벤치마크 셋	LLM API, 프롬프트 설계, JSON Schema 검증, 서버리스 프록시, 캐싱
렌더러·UX 담당	Player + 프리미티브 + UI + 시각 언어	React, SVG, GSAP Timeline, Monaco, 모션 디자인

- 스키마 3개는 공동 소유하되, **스키마·fixture 계약의 최종 버전 관리자(maintainer) 1인을 지정한다** (LLM·대본 담당 겸임 권장)
- 렌더러·UX 담당의 구현량이 가장 크므로 프리미티브 일부는 처음부터 다른 담당에게 배정한다
- fixture 파일이 모듈 간 접착제 — 서로를 기다리지 않고 병렬 개발한다

10. 리스크와 대응

리스크	대응
[1순위] Tracer 상태 직렬화 — locals/heap/mutation은 실행 후이 자동으로 주지 않음	최우선 스파이크로 Go/No-Go. settrace 대안 확보. 귀속 규칙·passive serializer를 계약에 선반영
[2순위] Untrusted client code 실행 — 사용자는 정의상 이해 못 한 코드를 실행함	Worker 무비밀, CSP/connect-src, JS bridge 제한 스파이크, 키 서버 측 보관
범용 상태 시각화 — 다양한 runtime state의 일관된 표현	프리미티브 5종 집중 + Generic State View + 층화 벤치마크로 커버리지 실측. generator·input 등은 정직하게 범위 밖 선언
코드 외부 전송 (Director 단계)	사용자 고지, opt-out, local-only 폴백
대기 시간과 사용 맥락의 충돌	콜드 스타트 선로딩, 단계별 P50/P95 예산 (기준치는 스파이크 후 고정)
LLM 대본 품질 미달 / "왜 AI가 필요한가"	폴백이 Core로 먼저 존재 + ablation으로 가치 정량 증명
Pyodide 제약	MVP 범위 명시로 기대치 관리, 실행기 독립 포맷으로 서버 실행기 확장 경로 확보
3인 범위 과대	증명 범위 축소, Slice 0/1/2 순차 통합, 통합·예외·테스트 비용을 기능보다 우선 배정

부록 A: 결정 기록

결정	선택	근거
타겟 순간	AI 코드 수령 직후 검증 (대표 시나리오)	복붙 문화라는 문제의식과 직결. 기술 범위는 Python 일반
범위	증명 범위(self-contained Python + 프리미티브 5종) / 확장 범위 분리. generator-input은 범위 밖	완성형과 MVP 혼재가 3인 완주의 최대 위협
LLM 역할	Core/Quality 분리 — Core는 LLM 없이 완주, Director는 품질 계층	값 신뢰성 + "왜 AI인가"의 논리 정합성
대본 참조	spanRef 중심 (LLM은 seq 숫자를 생성하지 않음) + 결정적 Resolver	실행 사실 생성 경로의 구조적 차단, 검증 단순화
narration	템플릿+바인딩, 관찰형 중심 / 해석형은 챗터 제목·요약에 한정	값 원천 원칙 강제 + 설명 가치 유지의 절충
실행 계층	Pyodide 우선, 하이브리드 확장	서버 측 RCE 위험·인프라 비용 대폭 감소
재생 모델	narrative-first 자동 재생 무비 + 일시정지 인스펙션	debugger-first(Python Tutor)·강의 영상 양쪽과 차별화
렌더러	MVP는 SVG+GSAP 단일, Pixi는 보류 자산	이중 렌더러 유지 비용 회피, 병목 실측 후 투입
평가	이해도 비교 실험(표본 수 선계산 후 설계 확정) + ablation	정량 근거 확보
KPI	지표 항목은 확정, 합격 수치는 스파이크 baseline 후 고정	근거 없는 숫자 배제
기존 프로토 타입	UI 레이아웃·색 체계 계승, Pixi 보류, tracing.ts(정규식 시물레이션) 폐기	화면이 코드에 대해 거짓말하는 구조는 목표와 상충

부록 B: 버전 이력

v1 → v2 (1차 외부 검토 반영): 리스크 1순위를 Tracer 상태 직렬화로 교체 · frameId/objectId/bounded serializer · narration 템플릿+바인딩 · 순서 검증 · DigestSpan 신설 · MVP/확장 분리(프리미티브 5종, SVG 단일) · Slice 0/1/2 · semantic timeline · 로딩 UX 모순 제거 · 표현 정직성 수정 · 외부 전송 고지 · 3중 실행 상한 · 캐시 키 확장 · transfer question · 스키마 maintainer 지정

v2 → v3 (2차 외부 검토 반영, 기획 동결): 1. 스텝 귀속 규칙 신설 — causedByLine/observedAtLine, "source-line boundary에서 관측되는 상태 변화"로 정직한 표현 2. localsDelta-reachableObjectsDelta로 개명, set/delete op 추가 (`del` 표현) 3. serializer에 observationally passive 원칙 추가 (repr/getattr 부작용 차단, UnsupportedObject 축약) 4. Screenplay를 stepRange에서 **spanRef 중심**으로 전환, 결정적 Resolver 모듈 신설 5. narration을 관찰형(트레이스 보장)/해석형(LLM 판단)으로 구분, 장면은 관찰형 중심 6. untrusted

code 위험 모델 신설, 리스크 2순위 등재 7. LLM의 Core/Quality 논리 정리 + **ablation 평가**(규칙 무비 vs LLM 무비) 추가 8. generator·input()을 범위와 벤치마크 양쪽에서 명시 제외 9. Generic State View 주장 범위 조정 ("지원 범위 내 최종 안전망") 10. KPI 수치는 스파이크 baseline 후 고정하기로 결정, 착수 순서(Slice 0 || Tracer 스파이크 즉시 병렬) 명시

v3 → v3.1: 지원 범위 매트릭스(2절) 추가 — 완전 지원(A) / Generic State View(B) / 제한 실행(C) / 범위 밖 안내(D)의 4단 판정을 코드 예시와 함께 명문화. 계약(스키마) 변경 없음 — 기획 동결 유지